

ONLINE BANKING MANAGEMENT SYSTEM

Mini-Project Reference Report

UE23CS352B – Object Oriented Analysis & Design

Course	UE23CS352B – Object Oriented Analysis & Design
Domain	Finance / Banking
Phase	Phase 2 – Analysis, Design & Implementation
Framework	Spring Boot (MVC) + Thymeleaf + MySQL
Language	Java 17+

TEAM MEMBERS			
Member	Name	Owned Major Use Case	Design Pattern
Student 1	Team Member 1	Account Management	Factory Method
Student 2	Team Member 2	Transaction & Fund Transfers	Facade Pattern
Student 3	Team Member 3	Loan Management	Observer Pattern
Student 4	Team Member 4	Admin Dashboard & Reports	Strategy Pattern

Academic Year 2025–2026

1. Problem Statement

The **Online Banking Management System (OBMS)** is a comprehensive web-based platform designed to digitize and automate core banking operations. Traditional banking involves extensive paperwork, long queues, and manual error-prone processes. The proposed system replaces these inefficiencies with a secure, scalable, and user-friendly digital solution.

The system serves three primary user roles: **Customers** who perform day-to-day banking activities such as account management, fund transfers, and loan applications; **Bank Employees** who manage customer accounts and process requests; and **Administrators** who oversee the entire system, manage staff, and generate operational reports.

Key challenges addressed include real-time transaction processing, secure multi-factor authentication, automated interest and EMI calculation, fraud detection alerts, and comprehensive audit trails for regulatory compliance.

2. Functional Requirements

2.1 Major Use Cases (4 — One Per Team Member)

#	Use Case	Owner	Description
UC-M 1	Account Management	Student 1	Full lifecycle: create, update, close accounts; view balance and download statements.
UC-M 2	Transaction & Fund Transfers	Student 2	Deposit, withdrawal, NEFT/IMPS transfers, transaction history with filters.
UC-M 3	Loan Management	Student 3	Apply, process, approve/reject loans; EMI schedule generation and repayment tracking.
UC-M 4	Admin Dashboard & Reports	Student 4	Employee management, system reports, audit logs, and operational analytics.

2.2 Minor Use Cases (4 — One Per Team Member)

#	Use Case	Owner	Description
UC-m 1	Profile & KYC Management	Student 1	Update personal details, upload KYC documents, change password and security settings.
UC-m 2	Bill Payments & Scheduled Transfers	Student 2	Pay utility bills, schedule recurring transfers, manage payees.
UC-m 3	Fixed Deposit Management	Student 3	Open/close FDs, view maturity date, auto-renewal settings.
UC-m 4	Notification & Alert System	Student 4	Email/SMS alerts for transactions, low balance, and loan due dates.

2.3 Detailed Functional Requirements

Customer Module

- User Registration: Aadhaar/PAN verification, OTP-based mobile verification, photo upload.
- Account Creation: Savings, Current, and Salary account types with auto-generated account numbers.
- Fund Transfer: NEFT, IMPS, and internal transfers with beneficiary management and PIN validation.
- Loan Application: Apply for Personal, Home, and Car loans; upload documents; track application status.
- Statement Download: Mini statement (last 10 txns) or full PDF statement for a selected date range.
- Fixed Deposits: Create FDs, view maturity calculator, set auto-renewal preferences.
- Bill Payments: Electricity, water, mobile recharge, and insurance premium payments.

Bank Employee Module

- Account Verification: Review and approve new account applications; flag incomplete KYC.
- Transaction Oversight: View all transactions under managed accounts; raise dispute tickets.
- Loan Processing: Review applications; approve/reject with remarks; trigger EMI schedule generation.
- Customer Support: Handle account unblock requests, dispute resolutions, and chequebook issuance.

Administrator Module

- Employee Management: Add, edit, deactivate bank employee accounts; assign branch and role.
- System Reports: Daily transaction summaries, NPA reports, and new account reports.
- Audit Logs: View complete audit trail of all system actions with timestamps and user IDs.
- Interest Rate Configuration: Set and update rates for savings, FDs, and loan products.
- Fraud Alert Management: View flagged transactions; block accounts suspected of fraud.

3. UML Diagrams

3.1 Use Case Diagram

The diagram shows interactions among three actors and the system's use cases. Draw using StarUML / draw.io and export as image.

Actor	Use Cases	Relationships
Customer	Register, Login, Manage Account, Transfer Funds, Apply Loan, Manage FD, Pay Bills, View Statements	<> Transfer Funds → Verify Balance <> Apply Loan → Upload Documents <> Login → 2FA
Bank Employee	Login, Verify Accounts, Process Loans, Handle Disputes, View Transactions	<> Process Loan → Review Documents
Administrator	Manage Employees, Configure Rates, Generate Reports, View Audit Logs, Manage Fraud Alerts	<> Generate Report → Download PDF

[Insert Use Case Diagram Image Here]

3.2 Class Diagram

Key classes, attributes, methods, and relationships are listed below.

Class	Key Attributes	Key Methods	Relationships
User	userId, name, email, passwordHash, role, isActive	login(), logout(), updateProfile()	Generalized by Customer, Employee, Admin
Customer	customerId, aadhaar, panNumber, kycStatus	applyForLoan(), openFD(), payBill()	Has many Account; extends User
Account	accountNumber, accountType, balance, ifscCode, status	deposit(), withdraw(), getStatement()	Belongs to Customer; has many Transaction
Transaction	txnId, amount, txnType, timestamp, status	process(), generateReceipt()	Belongs to Account
Loan	loanId, loanType, principal, interestRate, tenure, status	calculateEMI(), generateSchedule()	Belongs to Customer; has many LoanRepayment
FixedDeposit	fdId, principal, interestRate, maturityDate, autoRenew	calculateMaturity(), renew()	Belongs to Account
BankEmployee	employeeId, branch, designation	verifyAccount(), processLoan()	Extends User
Admin	adminId, accessLevel	generateReport(), configureRates()	Extends User; manages all entities
AuditLog	logId, action, performedBy, timestamp	record(), filter()	Associated with User and all entities

[Insert Class Diagram Image Here]

3.3 Activity Diagrams (4 Required)

Activity Diagram 1 – Fund Transfer (UC-M2)

Step	Actor	Action / Decision
1	Customer	Log in and navigate to Fund Transfer section.
2	System	Verify session token and 2FA status.
3	Customer	Enter beneficiary account, amount, and remarks.
4	System	Decision: Sufficient balance? No → Show error. Yes → Continue.
5	System	Validate beneficiary via IFSC + account number.
6	Customer	Enter transaction PIN.
7	System	Debit sender account; Credit receiver account atomically.
8	System	Generate Transaction ID; send SMS/email to both parties.
9	System	Update audit log; display success confirmation with receipt.

[Insert Activity Diagram 1 Image Here]

Activity Diagram 2 – Loan Application & Approval (UC-M3)

Step	Actor	Action / Decision
1	Customer	Navigate to Loan section; select loan type (Personal/Home/Car).
2	Customer	Fill application: amount, tenure, purpose, income details.
3	Customer	Upload KYC documents (salary slip, IT returns, address proof).
4	System	Validate documents; run automated credit score check.
5	System	Decision: Credit score sufficient? No → Auto-reject. Yes → Forward to Employee.
6	Employee	Review application; perform manual due diligence.
7	Employee	Decision: Approve (set rate, disburse) or Reject (with remarks).
8	System	Generate loan account; calculate EMI schedule.
9	System	Disburse loan amount to linked account; notify customer.

[Insert Activity Diagram 2 Image Here]

Activity Diagram 3 – Account Opening (UC-M1)

Step	Actor	Action / Decision
1	Customer	Register on portal with mobile OTP verification.
2	Customer	Select account type and fill personal details.
3	Customer	Upload KYC documents (Aadhaar, PAN, photo, address proof).
4	System	Validate documents; check for duplicate accounts via Aadhaar.
5	Employee	Review and approve/reject application; flag incomplete KYC.
6	System	Generate unique account number and IFSC code.
7	System	Set initial balance; create virtual debit card; send welcome email.
8	Customer	Activate account via link in email.

[Insert Activity Diagram 3 Image Here]

Activity Diagram 4 – Report Generation (UC-M4)

Step	Actor	Action / Decision
1	Admin	Log in to Admin Dashboard; navigate to Reports module.
2	Admin	Select report type: Transaction Summary / NPA / New Accounts / Employee Activity.
3	Admin	Apply filters: date range, branch, account type.
4	System	Query database with applied filters; aggregate results.
5	System	Render report in tabular and graphical format.
6	Admin	Option: Download as PDF or CSV; schedule automatic report via email.
7	System	Log report generation action in audit trail.

[Insert Activity Diagram 4 Image Here]

3.4 State Diagrams (4 Required)

State Diagram 1 – Account Lifecycle

From State	Trigger Event	To State	Guard Condition
PENDING	Application submitted	UNDER_REVIEW	KYC documents uploaded
UNDER_REVIEW	Employee approves	ACTIVE	All documents verified
UNDER_REVIEW	Employee rejects	REJECTED	Missing/invalid documents
ACTIVE	Suspicious transaction	TEMPORARILY_BLOCKED	Fraud detection trigger
TEMPORARILY_BLOCKED	Admin clears flag	ACTIVE	Investigation completed
ACTIVE	Customer/Admin closes	CLOSED	Zero balance, no pending EMIs
CLOSED	—	(Final State)	No further transitions allowed

[Insert State Diagram 1 Image Here]

State Diagram 2 – Loan Application Lifecycle

From State	Trigger Event	To State	Guard Condition
DRAFT	Customer saves progress	DRAFT	Form incomplete
DRAFT	Customer submits form	SUBMITTED	All mandatory fields filled
SUBMITTED	System runs credit check	UNDER_REVIEW	CIBIL score fetched
UNDER_REVIEW	Credit score too low	AUTO_REJECTED	Score < threshold
UNDER_REVIEW	Employee approves	APPROVED	Due diligence passed
UNDER_REVIEW	Employee rejects	REJECTED	Insufficient eligibility
APPROVED	Amount disbursed	ACTIVE	Loan account created
ACTIVE	All EMIs paid	CLOSED	Outstanding = 0
ACTIVE	3+ EMIs overdue	NPA	EMI missed consecutively

[Insert State Diagram 2 Image Here]

State Diagram 3 – Transaction Lifecycle

From State	Trigger Event	To State	Guard Condition
INITIATED	User submits request	VALIDATING	Request received
VALIDATING	Balance check passes	PROCESSING	Sufficient funds
VALIDATING	Balance insufficient / wrong PIN	FAILED	Validation error
PROCESSING	Bank confirms debit	COMPLETED	Atomic debit-credit success
PROCESSING	Network timeout	PENDING	Retry window open (30 min)
PENDING	Retry succeeds	COMPLETED	Second attempt successful
PENDING	Retry window expires	FAILED	Auto-reversal triggered
COMPLETED	Customer raises dispute	DISPUTED	Within 7 days
DISPUTED	Resolution passed	REVERSED	Chargeback approved

[Insert State Diagram 3 Image Here]

State Diagram 4 – Fixed Deposit Lifecycle

From State	Trigger Event	To State	Guard Condition
CREATED	Customer opens FD	ACTIVE	Minimum deposit met
ACTIVE	Premature withdrawal	PREMATURE_CLOSE D	Penalty deducted from interest
ACTIVE	Maturity reached, no auto-renew	MATURED	Tenure completed
ACTIVE	Maturity reached, auto-renew ON	RENEWED	Auto-renewal flag = true
MATURED	Amount transferred to savings	CLOSED	Principal + interest credited
RENEWED	New tenure begins	ACTIVE	New FD ID generated

[Insert State Diagram 4 Image Here]

4. Architecture Pattern – MVC

The system follows the **Model-View-Controller (MVC)** architectural pattern, implemented using the **Spring MVC framework**. This cleanly separates concerns and makes the codebase maintainable and testable.

Layer	Technology	Responsibility	OBMS Examples
Model	Spring Data JPA + Hibernate + MySQL	Represents domain entities and business logic. Handles DB interaction via JPA repositories.	Account, Transaction, Loan, Customer, AuditLog entities
View	Thymeleaf / JavaFX	Renders the UI. Thymeleaf templates populated by Controller with Model data.	dashboard.html, transfer.html, loan-apply.html, admin-reports.html
Controller	Spring MVC @Controller	Handles HTTP requests, invokes Service layer, returns view name + model attributes.	AccountController, TransactionController, LoanController, AdminController
Service	Spring @Service	Contains business logic. Called by Controller; calls Repository. Keeps Controller thin.	TransactionService, LoanService, NotificationService, FraudDetectionService
Repository	Spring Data JPA @Repository	Abstracts all database queries using JPQL or native SQL.	AccountRepository, TransactionRepository, LoanRepository

4.1 Request Flow

1. Browser sends HTTP request (e.g., POST /transfer/initiate).
2. DispatcherServlet routes to TransactionController.initiateTransfer().
3. Controller validates request data via @Valid and BindingResult.
4. Controller calls TransactionService.processTransfer(transferDto).
5. Service performs business logic: balance check, PIN validation, debit/credit.
6. Service calls AccountRepository and TransactionRepository to persist changes.
7. Service calls NotificationService to send SMS/email.
8. Controller receives result and returns view name 'transfer/success' with model.
9. Thymeleaf renders the HTML response and sends it back to the browser.

5. Design Patterns

Each team member owns one design pattern. The project uses 4 patterns covering **Creational**, **Structural**, and **Behavioral** categories, in addition to the MVC pattern enforced by the Spring framework.

5.1 Factory Method Pattern (Creational) – Student 1

Intent

Define an interface for creating an object, but let subclasses decide which class to instantiate. In OBMS, different account types have different rules for minimum balance, interest rate, and statement format.

How It Is Applied

An **AccountFactory** interface defines `createAccount()`. Concrete factories (`SavingsAccountFactory`, `CurrentAccountFactory`, `SalaryAccountFactory`) each implement this to return the correct Account subtype. The `AccountController` calls `AccountFactory.createAccount(type)` without knowing the underlying class.

Component	Class Name	Role
Interface	AccountFactory	Declares <code>createAccount(Customer, AccountType): Account</code>
Concrete Creator 1	SavingsAccountFactory	Creates <code>SavingsAccount</code> — 4% interest, Rs.1,000 min balance
Concrete Creator 2	CurrentAccountFactory	Creates <code>CurrentAccount</code> — 0% interest, Rs.10,000 min balance
Concrete Creator 3	SalaryAccountFactory	Creates <code>SalaryAccount</code> — 0% interest, no min balance
Product	Account	Abstract base class for all account types

```
// AccountFactory.java
public interface AccountFactory {
    Account createAccount(Customer customer, AccountType type);
}

// SavingsAccountFactory.java
@Component
public class SavingsAccountFactory implements AccountFactory {
    @Override
    public Account createAccount(Customer customer, AccountType type) {
        Account acc = new Account();
        acc.setAccountType(AccountType.SAVINGS);
        acc.setInterestRate(4.0);
        acc.setMinimumBalance(1000.0);
        acc.setCustomer(customer);
        return acc;
    }
}

// AccountService.java (usage)
```

```
public Account openAccount(Customer c, AccountType type) {
    AccountFactory factory = factoryMap.get(type); // injected Map
    Account account = factory.createAccount(c, type);
    return accountRepository.save(account);
}
```

5.2 Facade Pattern (Structural) – Student 2

Intent

Provide a unified interface to a set of interfaces in a subsystem. Fund transfer involves multiple services: balance validation, fraud detection, transaction recording, notification, and audit logging. The Facade simplifies this for the Controller.

How It Is Applied

TransactionFacade aggregates calls to **BalanceService**, **FraudDetectionService**, **NotificationService**, **AuditService**, and **TransactionService**. The Controller calls only **transferFunds()** on the Facade.

Component	Class Name	Role
Facade	TransactionFacade	Exposes transferFunds(TransferRequest): TransactionResult
Subsystem 1	BalanceValidationService	Checks sufficient balance and daily limit
Subsystem 2	FraudDetectionService	Flags suspicious transaction patterns
Subsystem 3	NotificationService	Sends SMS/email to sender and receiver
Subsystem 4	AuditService	Records the complete transaction audit entry
Client	TransactionController	Calls only Facade; unaware of subsystems

```
// TransactionFacade.java
@Service
public class TransactionFacade {
    @Autowired private BalanceValidationService balanceService;
    @Autowired private FraudDetectionService fraudService;
    @Autowired private NotificationService notificationService;
    @Autowired private AuditService auditService;
    @Autowired private TransactionService transactionService;

    public TransactionResult transferFunds(TransferRequest req) {
        balanceService.validateSufficientFunds(req);
        fraudService.checkForAnomalies(req);
        Transaction txn = transactionService.execute(req);
        notificationService.sendTransferNotification(txn);
        auditService.log(txn);
        return new TransactionResult(txn.getTxnId(), txn.getStatus());
    }
}

// TransactionController.java
```

```
@PostMapping('/transfer')
public String transfer(@Valid TransferRequest req, Model model) {
    TransactionResult result = transactionFacade.transferFunds(req);
    model.addAttribute('result', result);
    return 'transfer/success';
}
```

5.3 Observer Pattern (Behavioral) – Student 3

Intent

Define a one-to-many dependency so that when one object changes state, all its dependents are notified automatically. When a loan EMI becomes overdue, multiple downstream systems must react.

How It Is Applied

LoanRepaymentService acts as the Subject. When an EMI is overdue, it publishes a **LoanEMIOverdueEvent**. Registered Observers — **SMSNotificationListener**, **EmailNotificationListener**, **PenaltyCalculatorListener**, **NPAFlagListener** — each react independently. New listeners can be added without modifying **LoanRepaymentService**.

Component	Class Name	Role
Subject Interface	LoanEventPublisher	Maintains observer list; notifies on EMI overdue
Observer Interface	LoanEventListener	onEMIOverdue(LoanEvent): void
Concrete Observer 1	SMSNotificationListener	Sends SMS reminder to customer
Concrete Observer 2	EmailNotificationListener	Sends email with EMI details and penalty info
Concrete Observer 3	PenaltyCalculatorListener	Calculates and adds late payment penalty to loan
Concrete Observer 4	NPAFlagListener	Flags loan as NPA if overdue > 90 days
Spring Event Bus	ApplicationEventPublisher	Spring's built-in event mechanism (implements pattern natively)

```
// LoanEMIOverdueEvent.java
public class LoanEMIOverdueEvent extends ApplicationEvent {
    private final Loan loan;
    private final int daysOverdue;
    // constructor + getters
}

// LoanRepaymentService.java (Publisher)
@Service
public class LoanRepaymentService {
    @Autowired private ApplicationEventPublisher publisher;

    public void checkAndMarkOverdue(Loan loan) {
        int days = calculateDaysOverdue(loan);
        if (days > 0) {
            publisher.publishEvent(new LoanEMIOverdueEvent(this, loan, days));
        }
    }
}

// SMSNotificationListener.java (Observer)
@Component
public class SMSNotificationListener {
```

```
@EventListener
public void onEMIOverdue(LoanEMIOverdueEvent event) {
    smsService.send(event.getLoan().getCustomer().getMobile(),
        'EMI overdue by ' + event.getDaysOverdue() + ' days.');
```

5.4 Strategy Pattern (Behavioral) – Student 4

Intent

Define a family of algorithms, encapsulate each, and make them interchangeable. Different report types require different data aggregation strategies. The Admin selects the report strategy at runtime.

How It Is Applied

ReportStrategy interface defines `generateReport(ReportFilter)`. Concrete strategies: `DailyTransactionReportStrategy`, `NPAReportStrategy`, `NewAccountReportStrategy`, `EmployeeActivityReportStrategy`. **ReportService** holds the current strategy and delegates to it.

Component	Class Name	Role
Strategy Interface	ReportStrategy	<code>generateReport(ReportFilter filter): ReportData</code>
Concrete Strategy 1	DailyTransactionReportStrategy	Aggregates all transactions for a given day by branch
Concrete Strategy 2	NPAReportStrategy	Identifies loans overdue > 90 days with outstanding amount
Concrete Strategy 3	NewAccountReportStrategy	Lists accounts opened in a date range with KYC status
Concrete Strategy 4	EmployeeActivityReportStrategy	Summarises actions per employee: accounts verified, loans processed
Context	ReportService	Holds strategy reference; calls <code>strategy.generateReport()</code>

```
// ReportStrategy.java
public interface ReportStrategy {
    ReportData generateReport(ReportFilter filter);
}

// NPAReportStrategy.java
@Component('npaReportStrategy')
public class NPAReportStrategy implements ReportStrategy {
    @Autowired private LoanRepository loanRepo;
    @Override
    public ReportData generateReport(ReportFilter filter) {
        List npaLoans = loanRepo.findByStatusAndOverdueDaysGreaterThan(LoanStatus.NPA, 90);
        return new ReportData(npaLoans, 'NPA Report');
    }
}
```

```
// ReportService.java (Context)
@Service
public class ReportService {
    @Autowired private Map strategyMap;

    public ReportData generate(String reportType, ReportFilter filter) {
        ReportStrategy strategy = strategyMap.get(reportType + 'Strategy');
        return strategy.generateReport(filter); // selected at runtime
    }
}
```

6. Design Principles (SOLID)

Principle	Owner	How Applied in OBMS
Single Responsibility Principle (SRP)	Student 1	Each class has one job: AccountService only manages accounts; NotificationService only sends alerts; AuditService only logs. Controllers contain no business logic.
Open/Closed Principle (OCP)	Student 2	ReportStrategy interface allows adding new report types (e.g., FraudReportStrategy) without modifying ReportService. AccountFactory allows new account types without changing the factory interface.
Liskov Substitution Principle (LSP)	Student 3	SavingsAccount, CurrentAccount, and SalaryAccount are all substitutable wherever Account is expected. Overriding methods do not violate the contract defined in the base Account class.
Dependency Inversion Principle (DIP)	Student 4	Controllers depend on Service interfaces, not concrete implementations. Spring @Autowired injects the concrete bean at runtime. TransactionController uses TransactionFacade interface only.

7. Technology Stack

Layer	Technology	Purpose
Backend Framework	Spring Boot 3.x + Spring MVC	Core application framework; MVC architecture; REST and web layer
ORM / Persistence	Spring Data JPA + Hibernate	Entity mapping, database abstraction, JPQL queries
Database	MySQL 8.x	Relational storage for all entities; ACID compliance for transactions
Security	Spring Security + JWT	Authentication, role-based access control, BCrypt password encryption
Frontend / View	Thymeleaf + Bootstrap 5	Server-side templating; responsive UI; form handling
Build Tool	Maven	Dependency management, build lifecycle
Version Control	Git + GitHub (public repo)	Source code management; collaboration
Testing	JUnit 5 + Mockito	Unit tests for Service layer; mock Repository beans
PDF Generation	iText 7 / Apache PDFBox	Generating account statements and loan schedules as PDF
Notifications	JavaMailSender + Twilio SDK	Email/SMS alerts and OTP delivery

8. Database Schema (Key Tables)


```
-- users table (base for all user types)
CREATE TABLE users (
  user_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('CUSTOMER','EMPLOYEE','ADMIN') NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- accounts table
CREATE TABLE accounts (
  account_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  account_number VARCHAR(18) UNIQUE NOT NULL,
  account_type ENUM('SAVINGS','CURRENT','SALARY') NOT NULL,
  balance DECIMAL(15,2) DEFAULT 0.00,
  interest_rate DECIMAL(5,2),
  min_balance DECIMAL(10,2),
  status ENUM('PENDING','ACTIVE','BLOCKED','CLOSED') DEFAULT 'PENDING',
  customer_id BIGINT NOT NULL,
  ifsc_code VARCHAR(11) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (customer_id) REFERENCES users(user_id)
);

-- transactions table
CREATE TABLE transactions (
  txn_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  amount DECIMAL(15,2) NOT NULL,
  txn_type ENUM('CREDIT','DEBIT','TRANSFER','BILL_PAYMENT') NOT NULL,
  status ENUM('INITIATED','PROCESSING','COMPLETED','FAILED','REVERSED'),
  from_account_id BIGINT,
  to_account_id BIGINT,
  description VARCHAR(255),
  txn_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (from_account_id) REFERENCES accounts(account_id),
  FOREIGN KEY (to_account_id) REFERENCES accounts(account_id)
);

-- loans table
CREATE TABLE loans (
  loan_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  loan_type ENUM('PERSONAL','HOME','CAR') NOT NULL,
  principal DECIMAL(15,2) NOT NULL,
  interest_rate DECIMAL(5,2) NOT NULL,
  tenure_months INT NOT NULL,
  emi_amount DECIMAL(10,2),
  status ENUM('DRAFT','SUBMITTED','UNDER_REVIEW','APPROVED',
```

```

'ACTIVE','CLOSED','NPA','REJECTED'),
customer_id BIGINT NOT NULL,
disbursed_to BIGINT,
applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (customer_id) REFERENCES users(user_id),
FOREIGN KEY (disbursed_to) REFERENCES accounts(account_id)
);

-- fixed_deposits table
CREATE TABLE fixed_deposits (
fd_id BIGINT PRIMARY KEY AUTO_INCREMENT,
principal DECIMAL(15,2) NOT NULL,
interest_rate DECIMAL(5,2) NOT NULL,
tenure_months INT NOT NULL,
maturity_date DATE NOT NULL,
auto_renew BOOLEAN DEFAULT FALSE,
status ENUM('ACTIVE','MATURED','PREMATURE_CLOSED','CLOSED','RENEWED'),
account_id BIGINT NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (account_id) REFERENCES accounts(account_id)
);

```

9. Project Structure

```

obms/
├── src/main/java/com/obms/
│   ├── config/ # Spring Security config, MVC config, Bean configs
│   ├── controller/ # AccountController, TransactionController,
│   │   └── # LoanController, AdminController, AuthController
│   ├── service/ # AccountService, TransactionFacade, LoanService,
│   │   └── # ReportService, NotificationService, AuditService,
│   │       └── # FraudDetectionService, BalanceValidationService
│   ├── repository/ # AccountRepository, TransactionRepository,
│   │   └── # LoanRepository, UserRepository, AuditLogRepository
│   ├── model/ # Account, Transaction, Loan, User, Customer,
│   │   └── # Employee, Admin, FixedDeposit, AuditLog
│   ├── dto/ # TransferRequest, LoanApplicationDTO, ReportFilter,
│   │   └── # AccountOpenDTO, TransactionResult, ReportData
│   ├── factory/ # AccountFactory, SavingsAccountFactory,
│   │   └── # CurrentAccountFactory, SalaryAccountFactory
│   ├── strategy/ # ReportStrategy, DailyTransactionReportStrategy,
│   │   └── # NPAREportStrategy, NewAccountReportStrategy
│   ├── observer/ # LoanEMIOverdueEvent, SMSNotificationListener,
│   │   └── # EmailNotificationListener, NPAFlagListener
│   ├── exception/ # GlobalExceptionHandler, InsufficientFundsException,
│   │   └── # AccountNotFoundException, FraudDetectedException
│   └── src/main/resources/

```

```
■ ■ ■ ■ templates/ # Thymeleaf HTML: dashboard, transfer, loan-apply,  
■ ■ # admin, reports, login, register  
■ ■ ■ ■ static/ # CSS (Bootstrap), JavaScript, images  
■ ■ ■ ■ application.yml # DB config, Spring Security, mail/SMS settings  
■ ■ ■ ■ src/test/java/ # JUnit 5 test classes per service  
■ ■ ■ ■ pom.xml # Maven dependencies  
■ ■ ■ ■ README.md # Setup, run instructions, default credentials
```

10. Individual Contributions

Member	Major Use Case	Minor Use Case	Design Pattern	Design Principle	Key Classes
Student 1	Account Management Create, view, close; statement generation	Profile & KYC KYC upload, profile edit, password change	Factory Method AccountFactory hierarchy	Single Responsibility One class – one job	AccountController, AccountService, SavingsAccountFactory, CurrentAccountFactory
Student 2	Transaction & Transfers Deposit, withdrawal, NEFT/IMPS, history	Bill Payments Utility bills, recurring transfers, payees	FacadePattern TransactionFacade	Open/Closed Extend without modify	TransactionController, TransactionFacade, BalanceValidationService, FraudDetectionService
Student 3	Loan Management Apply, process, approve, EMI schedules	Fixed Deposits Open FD, auto-renewal, maturity tracking	Observer Pattern Loan event listeners	Liskov Substitution Subtype substitutability	LoanController, LoanService, LoanRepaymentService, EMIScheduleGenerator
Student 4	Admin Dashboard Employee mgmt, reports, audit logs	Notification Alerts Email/SMS for txn alerts, low balance, EMI due	Strategy Pattern Report generation	Dependency Inversion Depend on abstractions	AdminController, ReportService, ReportStrategy impls, AuditService, NotificationService

11. Screenshots Guide (White Background)

All screenshots must use a **white background** with input values populated and output clearly visible.

#	Screen	What to Show
1	Login Page	Email + password filled; role selector shown
2	Customer Dashboard	Account balance, recent transactions, quick action tiles
3	Fund Transfer – Form	Beneficiary details filled, amount entered, remarks added
4	Fund Transfer – Success	Transaction ID, amount, receiver, timestamp shown
5	Loan Application Form	Loan type selected, amount, tenure, documents uploaded
6	Loan Status Page	Application status, credit score result, employee remarks
7	EMI Schedule	Table with date, principal, interest, balance for all months
8	Account Statement	Date-range filtered transaction table; Download PDF button
9	Fixed Deposit – Open	Amount, tenure, interest rate, projected maturity amount
10	Bill Payment	Biller selected, amount, account debited, success message

#	Screen	What to Show
11	Admin Dashboard	Total accounts, transactions today, NPA count, charts
12	Report – Daily Transactions	Filtered report table with Download CSV option
13	Employee – Loan Processing	Application details, approve/reject buttons, remarks field
14	Audit Log	Action, user, timestamp, entity ID in table

12. GitHub Repository

Repository URL	https://github.com//obms-ooad-project
Visibility	Public – must be accessible to evaluator
Branch Structure	main (production), develop (integration), feature/* (per use case)

README.md must include:

- Project title, description, and domain.
- Prerequisites: Java 17+, Maven 3.8+, MySQL 8.
- Database setup: SQL script to create schema and seed data.
- How to run: mvn spring-boot:run; access at <http://localhost:8080>.
- Default login credentials for testing (Customer, Employee, Admin).
- Team member names and individual contribution summary.

13. Marks Distribution Checklist

Component	Marks	What Evaluator Checks
Analysis & Design Models (Use Case, Class, Activity x4, State x4)	2	1 Use Case Diagram with all actors + include/extend; 1 Class Diagram with 9+ classes; 4 Activity Diagrams (one per major use case); 4 State Diagrams (Account, Loan, Transaction, FD).
MVC Architecture Pattern	2	Clear separation: Controller → Service → Repository. No business logic in Controller. Thymeleaf views do not directly call Repository. @Controller, @Service, @Repository correctly used.
Design Patterns (4 total) + Design Principles (4 total)	3	At least 1 Creational + 1 Structural + 1 Behavioral + 1 additional (excluding Spring MVC). Code must show the implementation. 4 SOLID principles applied, each owned by one student.
Presentation / Demo / Code	3	Live demo of all 8 use cases. Each student explains their use case end-to-end: UI → Controller → Service → Repository → DB. Code walkthrough shows the design pattern applied.
TOTAL	10	All components present in PDF report and GitHub repository.

14. References

- [1] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [2] Spring Framework Documentation — <https://docs.spring.io/spring-framework/reference/>
- [3] Spring Data JPA Reference — <https://docs.spring.io/spring-data/jpa/reference/>
- [4] Thymeleaf Documentation — <https://www.thymeleaf.org/documentation.html>
- [5] Martin, R. C. (2002). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall.
- [6] MySQL 8.0 Reference Manual — <https://dev.mysql.com/doc/refman/8.0/en/>
- [7] JUnit 5 User Guide — <https://junit.org/junit5/docs/current/user-guide/>

END OF MINI-PROJECT REFERENCE REPORT · UE23CS352B – OOAD · Online Banking Management System